

VVP stands for Verilog Virtual Processor

VCD stands for Value Change Dump.

Device Under Test

example.v

```
module example (a,b,c,d,e,f,Y);  
    input a,b,c,d,e,f;  
    output Y;  
    wire t1,t2,t3;  
    nand G1(t1,a,b);  
    and G2(t2,c,~b,d);  
    nor G3(t3,e,f);  
    nand G4(Y,t1,t2,t3);  
endmodule
```

example_tb.v

```
`include "example.v"  
module example_tb;  
    reg a,b,c,d,e,f;  
    wire Y;  
    example DUT(a,b,c,d,e,f,Y);  
    initial begin  
        $dumpfile("example.vcd");  
        $dumpvars(0,example_tb);  
  
        $monitor($time,"a=%b,b=%b,c=%b,d=%b,e=%b,f=%b,Y=%b",a,b,c,d,e  
,f,Y);  
    end  
endmodule
```

```

        a=1;b=0;c=0;d=1;e=0;f=0;
        #5
        a=0;b=0;c=1;d=1;e=0;f=0;
        #5
        a=1;c=0;
        #5
        f=1;
        #5
        $finish;
    end
endmodule

```

Compile: iverilog -o example.vvp example.v

Simulate: vvp example.vvp

Execute: gtkwave example.vcd

Mux

```

module mux4to1 (D0, D1, D2, D3, S0, S1, Y);
    input D0, D1, D2, D3, S0, S1;
    output Y;

    // Internal wires for the inverted select signals
    wire not_S0, not_S1;

    // Internal wires for the outputs of the AND gates
    wire t0, t1, t2, t3;

    // Invert the select lines
    not u1(not_S0, S0);
    not u2(not_S1, S1);

```

```

    // AND gates to decode the select lines and pass the correct data
    and a0(t0, D0, not_S1, not_S0); // Active when S1=0, S0=0
    and a1(t1, D1, not_S1, S0);    // Active when S1=0, S0=1
    and a2(t2, D2, S1, not_S0);    // Active when S1=1, S0=0
    and a3(t3, D3, S1, S0);        // Active when S1=1, S0=1

    // OR gate to combine the results into the final output
    or o1(Y, t0, t1, t2, t3);

endmodule

```

mux_tb.v

```

`include "mux4to1.v"

module mux4to1_tb;
    // Registers for inputs, wire for output
    reg D0, D1, D2, D3, S0, S1;
    wire Y;

    // Instantiate the 4:1 MUX
    mux4to1 DUT(D0, D1, D2, D3, S0, S1, Y);

    initial begin
        // Setup GTKWave dump files
        $dumpfile("mux4to1.vcd");
        $dumpvars(0, mux4to1_tb);

        // Monitor changes in the terminal
        $monitor($time, " S1=%b, S0=%b | D0=%b, D1=%b, D2=%b, D3=%b |
Y=%b", S1, S0, D0, D1, D2, D3, Y);

        // --- THE TIMELINE ---
    end

```

```

// Time 0: Set a fixed data pattern: 1, 0, 1, 0
D0=1; D1=0; D2=1; D3=0;

// Select D0 (S1=0, S0=0). Output Y should be 1.
S1=0; S0=0;
#5;

// Select D1 (S1=0, S0=1). Output Y should be 0.
S1=0; S0=1;
#5;

// Select D2 (S1=1, S0=0). Output Y should be 1.
S1=1; S0=0;
#5;

// Select D3 (S1=1, S0=1). Output Y should be 0.
S1=1; S0=1;
#5;

// End simulation
$finish;

end
Endmodule

```

Homework: Design Full adder using HDL.